



TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

Bài 8

Sắp xếp kết quả truy vấn

Thái Nguyên, tháng 8 năm 2024



Nội dung chính

1. Sắp xếp kết quả truy vấn
2. Phép kết nối
3. Thống kê dữ liệu với GROUP BY
4. Mệnh đề điều kiện đối với hàm nhóm - HAVING



8.1. Sắp xếp kết quả truy vấn

- Mặc định các dòng dữ liệu trong kết quả của câu truy vấn tuân theo thứ tự của chúng trong bảng dữ liệu hoặc được sắp xếp theo chỉ mục (nếu trên bảng có chỉ mục).
- Trong trường hợp muốn dữ liệu được sắp xếp theo **chiều tăng hoặc giảm** của giá trị của một hoặc nhiều trường, ta sử dụng thêm mệnh đề **ORDER BY** trong câu lệnh **SELECT**.
- Sau **ORDER BY** là danh sách các cột cần sắp xếp (tối đa là 16 cột). Dữ liệu được sắp xếp có thể theo **chiều tăng (ASC)** hoặc **giảm (DESC)**, mặc định là sắp xếp theo chiều tăng.



8.1. Sắp xếp kết quả truy vấn

Cú pháp của **ORDER BY**

- **SELECT** [Danh sách các cột]
- **FROM** [Tên bảng]
- **WHERE** [Điều kiện lọc (nếu có)]
- **ORDER BY** [Tên cột1] [**ASC**|**DESC**], [Tên cột2] [**ASC**|**DESC**], ...;
- Trong đó:
 - **ASC**: Thứ tự tăng dần (mặc định).
 - **DESC**: Thứ tự giảm dần.



8.1. Sắp xếp kết quả truy vấn

➤ **Chú ý:** Nếu sau ORDER BY có nhiều cột thì việc sắp xếp dữ liệu sẽ được ưu tiên theo chiều từ trái qua phải.

➤ **Ví dụ Cho CSDL Quản lý bán hàng:**

1. Câu lệnh dưới đây hiển thị danh sách các hoá đơn và sắp xếp theo chiều giảm dần của số lượng sản phẩm

```
SELECT * FROM ChiTietHoaDon
```

```
ORDER BY SoLuong DESC
```

2. Liệt kê danh sách khách hàng và sắp xếp tên khách theo Alphabet, nếu trùng tên thì sắp theo địa chỉ

```
SELECT * FROM Khach
```

```
ORDER BY TenK, DiaChi
```



8.1. Sắp xếp kết quả truy vấn

Kết hợp ORDER BY với WHERE

- **Yêu cầu:** Lấy danh sách các chi tiết hóa đơn với số lượng lớn hơn 10 và sắp xếp theo đơn giá (DonGia) từ cao đến thấp.
- Câu lệnh SQL:

SELECT SoHD, MaH, SoLuong, DonGia

FROM ChiTietHoaDon

WHERE SoLuong > 10

ORDER BY DonGia DESC;

- **Giải thích:**
- WHERE SoLuong > 10: Lọc những chi tiết hóa đơn có số lượng lớn hơn 10.
- ORDER BY DonGia DESC: Sắp xếp kết quả theo đơn giá từ cao đến thấp.



8.2. Phép kết nối

- Trong cơ sở dữ liệu quan hệ, việc lấy dữ liệu từ nhiều bảng có liên quan là một thao tác phổ biến. Để thực hiện điều này, chúng ta sử dụng phép kết nối (JOIN) giữa các bảng.
- Có hai cách chính để thực hiện kết nối:
 - Sử dụng điều kiện kết nối bằng trong mệnh đề WHERE.
 - Sử dụng cú pháp JOIN trong mệnh đề FROM.



8.2. Phép kết nối

- Khi cần thực hiện một yêu cầu truy vấn dữ liệu từ **hai hay nhiều bảng**, ta phải sử dụng đến phép kết nối. Để thực hiện được một phép nối, cần phải xác định được những yếu tố sau:
 - ☐ Những cột nào cần hiển thị trong kết quả truy vấn.
 - ☐ Những bảng nào có tham gia vào truy vấn.
 - ☐ Điều kiện để thực hiện phép nối giữa các bảng dữ liệu là gì?
- Trong các yếu tố kể trên, việc xác định chính xác điều kiện để thực hiện phép nối giữa các bảng đóng vai trò quan trọng nhất. Trong đa số các trường hợp, điều kiện của phép nối được xác định nhờ vào mối quan hệ giữa các bảng cần phải truy xuất dữ liệu.
- Thông thường, đó là **điều kiện bằng nhau giữa khoá chính và khóa ngoài của hai bảng** có quan hệ với nhau.



8.2. Phép kết nối

- Câu lệnh dưới đây hiển thị danh sách các khách hàng đã mua hàng: Mã khách, tên khách, số hoá đơn và ngày hóa đơn

SELECT *Khach.MaK, TenK', SoHD, Ngay* -- *Cột hiển thị* --

FROM *Khach, HoaDon* -- *Bảng tham gia* --

WHERE *Khach.MaK = HoaDon.MaK* -- *ĐK kết nối* --



8.2. Phép kết nối

- Tên của một số cột nào đó trong các bảng có tham gia vào truy vấn. Nếu tên cột trong các bảng trùng tên nhau thì tên cột phải được viết dưới dạng: ***Tên_bảng.tên_cột***
- Dấu sao (*) được sử dụng trong danh sách chọn khi cần hiển thị tất cả các cột của các bảng tham gia truy vấn.
- Trong trường hợp cần hiển thị tất cả các cột của một bảng nào đó, ta sử dụng cách viết: ***tên_bảng.****
- **Ví dụ:** Liệt kê danh sách khách hàng đã mua hàng có địa chỉ tại 'Thái Nguyên'

```
SELECT  Khách.MaK, Khách.TenK,  
        Khách.DiaChi, Khách.SDT  
FROM    Khách, HoaDon  
WHERE   Khách.MaK = HoaDon.MaK  
AND     Khách.DiaChi = N'Thái Nguyên'
```

hoặc viết dưới dạng ngắn gọn hơn:

```
SELECT  Khách.*  
FROM    Khách, HoaDon  
WHERE   Khách.MaK = HoaDon.MaK  
AND     Khách.DiaChi = N'Thái Nguyên'
```



Phép kết nối

- **Ví dụ:** Liệt kê danh sách các sinh viên tham gia thi môn 'Cơ sở dữ liệu'

```
SELECT SinhVien.MaSV, HoDem, Ten,  
MaLop, NgaySinh, GioiTinh, DiaChi  
FROM SinhVien , MonHoc , Diem  
WHERE SinhVien.MaMon = Diem.MaMon  
AND MonHoc.MaMon=Diem.MaMon  
AND TenMon='Cơ sở dữ liệu'
```

```
SELECT SinhVien.*  
FROM SinhVien , MonHoc , Diem  
WHERE SinhVien.MaMon = Diem.MaMon  
AND MonHoc.MaMon=Diem.MaMon  
AND TenMon='Cơ sở dữ liệu'
```



Sử dụng phép kết nối (JOIN) trong SQL server

- **Phép kết nối (JOIN)** là cách kết hợp dữ liệu từ nhiều bảng dựa trên một điều kiện cụ thể. Trong hệ quản trị CSDL, các bảng thường có mối quan hệ với nhau qua các khóa (key), và phép kết nối cho phép truy xuất dữ liệu từ nhiều bảng liên quan.
- **Cú pháp JOIN** cho phép kết nối các bảng một cách rõ ràng và linh hoạt hơn.
- **Các loại JOIN phổ biến:**
 - INNER JOIN
 - LEFT JOIN
 - RIGHT JOIN
 - FULL OUTER JOIN



Sử dụng phép kết nối (JOIN) trong SQL server

- **Cú pháp tổng quát:**

SELECT [Danh sách cột]

FROM Bảng1

[LOẠI JOIN] Bảng2 ON Bảng1.Cột_Kết_Nối = Bảng2.Cột_Kết_Nối;



Sử dụng phép kết nối (JOIN) trong SQL server

- **Ví dụ:** Lấy danh sách các hóa đơn cùng với tên khách hàng đã mua hàng.
- **Sử dụng INNER JOIN:**

```
SELECT HoaDon.SoHD, HoaDon.Ngay, Khách.TenK  
FROM HoaDon  
INNER JOIN Khách ON HoaDon.MaK = Khách.MaK;
```
- **Giải thích:**
 - FROM HoaDon INNER JOIN Khách: Thực hiện INNER JOIN giữa HoaDon và Khách.
 - ON HoaDon.MaK = Khách.MaK: Điều kiện kết nối.



INNER JOIN

- **Định nghĩa:** Trả về các bản ghi khi có sự khớp dữ liệu trong cả hai bảng. Nếu không có bản ghi nào khớp, thì nó sẽ không trả về gì.
- **Cú pháp:**

SELECT column_name(s)

FROM table1

INNER JOIN table2 **ON** table1.column_name = table2.column_name;



INNER JOIN

- **Ví dụ:** Lấy thông tin chi tiết về các hóa đơn, bao gồm số hóa đơn, ngày, tên khách hàng, tên hàng, số lượng và đơn giá.

```
SELECT HoaDon.SoHD, HoaDon.Ngay, Khach.TenK, Hang.TenH,  
ChiTietHoaDon.SoLuong, ChiTietHoaDon.DonGia
```

```
FROM HoaDon
```

```
INNER JOIN Khach ON HoaDon.MaK = Khach.MaK
```

```
INNER JOIN ChiTietHoaDon ON HoaDon.SoHD = ChiTietHoaDon.SoHD
```

```
INNER JOIN Hang ON ChiTietHoaDon.MaH = Hang.MaH;
```



LEFT JOIN (LEFT OUTER JOIN)

- Định nghĩa: Trả về tất cả các bản ghi từ bảng bên trái và các bản ghi khớp từ bảng bên phải. Nếu không có giá trị khớp từ bảng bên phải, kết quả sẽ là NULL cho cột của bảng bên phải.
- Cú pháp:
SELECT column_name(s)
FROM table1
LEFT JOIN table2 **ON** table1.column_name = table2.column_name;



LEFT JOIN (LEFT OUTER JOIN)

- **Ví dụ:** Lấy danh sách tất cả các khách hàng và hóa đơn của họ. Nếu khách hàng chưa có hóa đơn, hiển thị NULL cho thông tin hóa đơn.
- **Câu lệnh SQL:**

```
SELECT Khách.MaK, Khách.TenK, HoaDon.SoHD, HoaDon.Ngay  
FROM Khách  
LEFT JOIN HoaDon ON Khách.MaK = HoaDon.MaK;
```
- **Giải thích:**
LEFT JOIN: Lấy tất cả các bản ghi từ bảng bên trái (Khách) và các bản ghi khớp từ bảng bên phải (HoaDon).



RIGHT JOIN (RIGHT OUTER JOIN)

- Định nghĩa: Trả về tất cả các bản ghi từ bảng bên phải và các bản ghi khớp từ bảng bên trái. Nếu không có giá trị khớp từ bảng bên trái, kết quả sẽ là NULL cho cột của bảng bên trái.
- Cú pháp:
SELECT column_name(s)
FROM table1
RIGHT JOIN table2 **ON** table1.column_name = table2.column_name;



RIGHT JOIN (RIGHT OUTER JOIN)

- **Ví dụ:** Lấy danh sách tất cả các hóa đơn và thông tin khách hàng. Nếu hóa đơn không có khách hàng tương ứng, hiển thị NULL cho thông tin khách hàng.

- **Câu lệnh SQL:**

SELECT HoaDon.SoHD, HoaDon.Ngay, Khach.MaK, Khach.TenK

FROM HoaDon

RIGHT JOIN Khach **ON** HoaDon.MaK = Khach.MaK;

- **Lưu ý:** Trong thực tế, RIGHT JOIN ít được sử dụng hơn LEFT JOIN. Câu truy vấn trên có thể được viết lại bằng LEFT JOIN bằng cách đổi thứ tự bảng.



FULL JOIN (FULL OUTER JOIN)

- **Định nghĩa:** Trả về tất cả các bản ghi khi có sự khớp ở một trong hai bảng. Nếu không có khớp trong một bảng, giá trị NULL sẽ được trả về cho cột tương ứng.
- **Cú pháp:**

SELECT column_name(s)

FROM table1

FULL JOIN table2 **ON** table1.column_name = table2.column_name;



FULL JOIN (FULL OUTER JOIN)

- **Ví dụ:** Lấy danh sách tất cả các khách hàng và hóa đơn, bao gồm cả những khách hàng chưa có hóa đơn và những hóa đơn không có khách hàng.

- **Câu lệnh SQL:**

SELECT Khách.MaK, Khách.TenK, HoaDon.SoHD, HoaDon.Ngay

FROM Khách

FULL OUTER JOIN HoaDon **ON** Khách.MaK = HoaDon.MaK;



CROSS JOIN

- **Định nghĩa:** Trả về tất cả các kết hợp có thể giữa các hàng của hai bảng. Kết quả là một tích Đề-các của các hàng trong hai bảng.
- **Cú pháp:**

SELECT column_name(s)

FROM table1

CROSS JOIN table2;



CROSS JOIN

- **Ví dụ:** Kết hợp tất cả sinh viên với tất cả các khóa học mà không cần điều kiện.

SELECT students.name, courses.course_name

FROM students

CROSS JOIN courses;



So sánh giữa Kết nối bằng trong WHERE và JOIN

Điểm Chung

- Cả hai phương pháp đều cho phép kết nối và lấy dữ liệu từ nhiều bảng.
- Kết quả truy vấn có thể giống nhau khi sử dụng INNER JOIN và điều kiện kết nối trong WHERE.



So sánh giữa Kết nối bằng trong WHERE và JOIN

Điểm khác biệt

- Trong các phiên bản hiện đại của SQL, cú pháp JOIN đã thay thế phần lớn việc sử dụng điều kiện kết nối bằng trong mệnh đề WHERE do những ưu điểm của nó:
 - Cú pháp JOIN rõ ràng hơn: Với cú pháp JOIN, chúng ta có thể dễ dàng phân biệt được điều kiện kết nối (nằm trong mệnh đề ON) và các điều kiện lọc dữ liệu (nằm trong mệnh đề WHERE).
 - Hỗ trợ các loại JOIN phức tạp: Cú pháp JOIN cho phép chúng ta thực hiện các loại kết nối phức tạp hơn như LEFT JOIN, RIGHT JOIN, và FULL JOIN, điều mà điều kiện kết nối bằng trong mệnh đề WHERE không hỗ trợ rõ ràng.



8.3. Thống kê dữ liệu với GROUP BY

Giới thiệu về GROUP BY

- Trong SQL, Mệnh đề GROUP BY trong câu lệnh SELECT được sử dụng để nhóm các bản ghi có cùng giá trị trong một hoặc nhiều cột lại với nhau và thực hiện các phép toán tổng hợp trên các nhóm đó.
- Khi sử dụng GROUP BY, chúng ta thường kết hợp với các hàm tổng hợp như SUM(), COUNT(), AVG(), MAX(), MIN() để tính toán dữ liệu trong mỗi nhóm.



8.3. Thống kê dữ liệu với GROUP BY

SQL cung cấp các hàm nhóm dưới đây:

Hàm nhóm	Chức năng
SUM(biểu_thức)	Tính tổng các giá trị
AVG(biểu_thức)	Tính trung bình của các giá trị
COUNT(biểu_thức)	Đếm số các giá trị trong biểu thức
COUNT(*)	Đếm số các dòng được chọn
MAX(biểu_thức)	Tính giá trị lớn nhất
MIN(biểu_thức)	Tính giá trị nhỏ nhất



8.3. Thống kê dữ liệu với GROUP BY

- **Cú pháp của GROUP BY**

SELECT [Danh sách các cột], [Hàm tổng hợp]

FROM [Tên bảng]

WHERE [Điều kiện lọc (nếu có)]

GROUP BY [Tên cột1], [Tên cột2], ...;

- **Trong đó:**

- Danh sách các cột: Cột hoặc các cột chúng ta muốn hiển thị trong kết quả.
- Hàm tổng hợp: Các hàm như SUM(), COUNT(), AVG(), MAX(), MIN(), v.v.
- **GROUP BY:** Cột hoặc các cột dùng để nhóm dữ liệu.



8.3. Thống kê dữ liệu với GROUP BY

Thống kê trên toàn bộ dữ liệu

- Khi cần tính toán giá trị thống kê trên toàn bộ dữ liệu, ta sử dụng các hàm nhóm trong danh sách chọn của câu lệnh SELECT. Trong trường hợp này, trong danh sách chọn không được sử dụng bất kỳ một tên cột hay biểu thức nào ngoài các hàm gộp.
- **Ví dụ:** Để thống kê giá trung bình của tất cả các hoá đơn trong bảng chi tiết hoá đơn ta sử dụng câu lệnh như sau:

```
SELECT AVG(DonGia)
```

```
FROM ChiTietHoaDon
```



8.3. Thống kê dữ liệu với GROUP BY

Thống kê dữ liệu trên các nhóm

- Trong trường hợp cần thực hiện tính toán các giá trị thống kê trên các nhóm dữ liệu, ta sử dụng mệnh đề GROUP BY để phân hoạch dữ liệu vào trong các nhóm. Các hàm nhóm được sử dụng sẽ thực hiện thao tác tính toán trên mỗi nhóm và cho biết giá trị thống kê theo các nhóm dữ liệu.
- **Ví dụ:** Câu lệnh dưới đây cho biết số lần mua hàng của mỗi khách hàng

```
SELECT Khách.MaK, Khách.TenK, COUNT(Khách.MaK) AS N'Số lần mua hàng'
```

```
FROM Khách, HoaDon
```

```
WHERE Khách.MaK = HoaDon.MaK
```

```
GROUP BY Khách.MaK, Khách.TenK
```



8.3. Thống kê dữ liệu với GROUP BY

Thống kê dữ liệu trên các nhóm

- **Chú ý:**

- Biểu thức nào điều khiển việc phân nhóm dữ liệu thì các biểu thức đó phải được liệt kê sau mệnh đề GROUP BY.
- Trong trường hợp danh sách chọn của câu lệnh SELECT có các hàm nhóm và những biểu thức không phải là đối số của các hàm nhóm thì những biểu thức này phải được liệt kê đầy đủ trong mệnh đề GROUP BY, nếu không câu lệnh sẽ không hợp lệ



8.4. Mệnh đề điều kiện đối với hàm nhóm - HAVING

Giới thiệu về HAVING

- Trong SQL, Mệnh đề HAVING được sử dụng để lọc dữ liệu sau khi đã thực hiện nhóm dữ liệu với GROUP BY và áp dụng các hàm tổng hợp như SUM(), COUNT(), AVG(), MAX(), MIN().
- Mệnh đề HAVING cho phép chúng ta đặt điều kiện cho các nhóm dữ liệu, giúp chúng ta chỉ giữ lại những nhóm thỏa mãn điều kiện cụ thể.
- Một điểm khác biệt giữa HAVING và WHERE là trong điều kiện của WHERE không được có các hàm nhóm trong khi HAVING lại cho phép sử dụng các hàm nhóm trong điều kiện của mình.



8.4. Mệnh đề điều kiện đối với hàm nhóm - HAVING

- **Cú pháp của HAVING**

SELECT [Danh sách các cột], [Hàm tổng hợp]

FROM [Tên bảng]

WHERE [Điều kiện lọc (nếu có)]

GROUP BY [Tên cột1], [Tên cột2], ...

HAVING [Điều kiện đối với hàm nhóm];

- **Trong đó:**

- Danh sách các cột: Cột hoặc các cột chúng ta muốn hiển thị trong kết quả.
- Hàm tổng hợp: Các hàm như SUM(), COUNT(), AVG(), MAX(), MIN(), v.v.
- GROUP BY: Cột hoặc các cột dùng để nhóm dữ liệu.
- HAVING: Điều kiện lọc áp dụng cho các nhóm dữ liệu.



8.4. Mệnh đề điều kiện đối với hàm nhóm - HAVING

Ví dụ về HAVING

- Sử dụng cơ sở dữ liệu quản lý bán hàng, gồm các bảng Hang, Khach, HoaDon, và ChiTietHoaDon.
- **Ví dụ 1: Đưa ra danh sách khách hàng có số lần mua hàng lớn hơn hoặc bằng 5.**

SELECT Khach.MaK, Khach.TenK, **COUNT**(Khach.MaK) **AS** N'Số lần mua hàng'

FROM Khach, HoaDon

WHERE Khach.MaK = HoaDon.MaK

GROUP BY Khach.MaK, Khach.TenK

HAVING COUNT(Khach.MaK) >= 5



HƯỚNG DẪN THỰC HÀNH



Cám ơn đã chú ý lắng nghe!